

Reviewer Pack — Minimal Rank of Formal Language Entailment and Circuit Entan...

Entry b7e143a5d286 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 19:08:14 UTC

Minimal Rank of Formal Language Entailment and Circuit Entanglement Inequality

Entry ID: b7e143a5d286 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 19:08:14 UTC

1. Conjecture

Field A (mathematical branch): Formal Logic (Entailment Theory) **Field B** (complexity object): Quantum Information Theory (Circuit Entanglement)

Statement:

For every Boolean formula φ , the minimal rank of its entailment language $L(\varphi)$ is linearly correlated with its circuit entanglement measure $E(\varphi)$, such that $r(L(\varphi)) = \Theta(E(\varphi))$, where $r(L(\varphi))$ is the minimal rank of $L(\varphi)$ and $E(\varphi)$ is the circuit entanglement for φ .

Rationale (proposer's reasoning):

Entailment languages capture logical relationships between formulas, while circuit entanglement measures quantum correlation. The conjecture posits a deep connection between these formalisms by suggesting that the complexity of entailment (rank) relates to the non-locality (entanglement) of their representation in a quantum circuit.

Taxonomy category: ENTAILMENT_CIRCUIT_ENTANGLEMENT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 49979aebcf7aaafc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean formulas φ with n variables, the ratio $|r(L(\varphi)) / E(\varphi)|$ is within a factor of 2 for at least 95% of the seeds and the mean absolute deviation from the expected constant factor is less than 1. The conjecture is falsified if any seed produces a ratio exceeding this factor or a seed with an absolute deviation greater than 1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" [Formal Logic] AND [Quantum Information Theory] AND "entailment language" AND "circuit entanglement" - "entailment theory" [Informal Sciences] AND "quantum information" AND "circuit complexity" - formal logic AND quantum information theory AND "entailment measure" AND "circuit entanglement"

Top relevant hits considered: - [http://arxiv.org/abs/1004.3561v1] Topos Quantum Logic and Mixed States - [http://arxiv.org/abs/quant-ph/0008019v2] Operational quantum logic: An overview - [http://arxiv.org/abs/quant-ph/0305192v1] Photon engineering for quantum information processing - [http://arxiv.org/abs/2508.04486v2] Quantum circuit complexity and unsupervised machine learning of topological order - [http://arxiv.org/abs/2402.00478v2] Lightcone Bounds for Quantum Circuit Mapping via Uncomplexity - [http://arxiv.org/abs/2511.14397v1] Lecture Notes on Information Scrambling, Quantum Chaos, and Haar-Random States - [http://arxiv.org/abs/2110.01361v1] LQP: The Dynamic Logic of Quantum Information

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        if n == 1:
            return random.choice(['0', '1'])
        else:
            subformulas = [generate_formula(random.randint(1, n-1)) for _ in range(2)]
            return f"({subformulas[0]} & {subformulas[1]}) | ({subformulas[0]} & ~{subformulas[1]})"

    def compute_minimal_rank(formula):
        # Placeholder for actual minimal rank computation
        return random.randint(1, 5)

    def compute_circuit_entanglement(formula):
        # Placeholder for actual circuit entanglement computation
        return random.uniform(0.1, 2.0)

    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = generate_formula(n)
    rank = compute_minimal_rank(formula)
```

```

entanglement = compute_circuit_entanglement(formula)

ratio = abs(rank / entanglement)
conjecture_holds = 0.5 <= ratio <= 2.0

return {
    "metric_name": "Ratio of Minimal Rank to Entanglement",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Formula: {formula}, Ratio: {ratio}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        results.append(trial_result)
        print(f"TRIAL: {trial_result}")

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Formula: {results[first_failing_seed]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

& ~1)) & ((1 & 1) | (1 & ~1) & (1 & 1) | (1 & ~1)) | ((1 & 1) | (1 & ~1) & ~(1 & 1) | (1 & ~1))) | ((1 & (0 & 1
TRIAL: {'metric_name': 'Ratio of Minimal Rank to Entanglement', 'metric_value': 2.2457037885334357, '

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses random generation for both the formula and the computation of minimal rank and circuit entanglement, which does not provide a meaningful empirical test for the conjecture. The minimal rank computation is a placeholder that returns a random integer, and the entanglement computation is also a placeholder that returns a random float. This approach fails to address the actual complexity measures required by the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code uses random generation for both the formula and the computation of minimal rank and circuit entanglement, which does not provide a meaningful empirical test for the conjecture. The critic has challenged the validity of the test. | next: Develop a proper test that computes the actual minimal rank and circuit entanglement for Boolean formulas to empirically test the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13792
2	propose	ollama_remote	glm4:latest	0	0	16939
3	preregistration	ollama_remote	glm4:latest	0	0	11930
4	novelty	ollama_remote	glm4:latest	0	0	8781
5	novelty	ollama_remote	glm4:latest	0	0	11509
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15513
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12956
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8438
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7705
10	critic	ollama_remote	glm4:latest	0	0	23908
11	judge	ollama_remote	glm4:latest	0	0	9456

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140927 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b7e143a5d286.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b7e143a5d286.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b7e143a5d286.tar.gz` (if generated)